

Addressing hallucinations in generative AI agents using observability and dual memory knowledge graphs

Amali Matharaarachchi ^a, Harsha Moraliyage ^a, Nishan Mills ^a, Gihan Gamage ^a,
Daswin De Silva ^{a,*}, Milos Manic ^b

^a La Trobe Artificial Intelligence Institute, La Trobe University, Melbourne, Australia

^b Department of Computer Science, Virginia Commonwealth University, 23284, Richmond, VA, USA

ARTICLE INFO

Keywords:

Hallucination
AI Observability
Knowledge graph
Generative AI agents
Agent reasoning
Retrieval augmented generation

ABSTRACT

Generative AI has rapidly progressed from chatbots and assistants to agents across diverse applications and domains. Generative AI agents demonstrate sophisticated operation through autonomy, tool use and decision making with minimal human input. Despite these performance gains, agents are still impacted by the foundational limitations of Generative AI models. Among these, hallucinations are a major limitation that affects agent operation in real-world settings, leading to risk and loss. Several recent work aim to address hallucinations through methods such as retrieval-augmented generation and reflection prompting, however, these only provide partial improvements. An effective yet underexplored approach is in the observability data generated by an agent in its deployed and operational settings. Drawing on agent observability data, this paper proposes a dual memory knowledge graph approach that integrates Semantic and Observability Memory to address hallucinations in Generative AI agents. Semantic Memory provides organized domain knowledge for precise factual grounding. Observability Memory transforms logs, traces, and execution results into agent validated planning histories. Hallucinations are then addressed by grounded planning in verified past interactions with known, reliable outcomes. This approach is evaluated in a two-stage experimental setup aligned with its dual memory design. Observability memory is evaluated on the HotpotQA dataset to assess its impact on reasoning grounding, using metrics that capture both factual accuracy and reasoning hallucinations. The SM3-Text-to-Query benchmark and Synthea-based medical QA datasets are used to assess factual grounding of the semantic memory. Results from both experiments demonstrate reductions in hallucinations, with semantic memory for contextual grounding reducing factual hallucinations, and observability memory for reasoning grounding reducing faithfulness hallucinations.

1. Introduction

Generative AI has achieved significant performance gains across diverse benchmarks [1] and effective application of automation and augmentation in diverse practical settings [2]. Despite this success, Generative AI is still impacted by the foundational limitations of lack of a world model, hallucinations and bias [3]. Hallucinations are a primary limitation as it affects downstream implementation of Generative AI such as in chatbots, assistants and agents. Hallucinations undermine human-AI trust and pose major risks in high-stakes application settings [4,5]. Hallucinations often arise from flawed reasoning, or the absence of verifiable memory. Although grounding responses in past interactions and verified knowledge is a widely practiced method of hallucination mitigation, doing so effectively is more complex [6]. Unlike standard

Generative AI applications that generate responses in a single forward pass, AI agent systems decompose user queries into structured subtasks executed iteratively. These agents rely on external tools, perform long horizon planning, and refine the output through feedback loops [7]. As a result, hallucination in agents can emerge from generation errors, inaccurate planning steps, poor memory recall, and misaligned decision sequences. This makes hallucination a multi-step failure rather than a single point error [8]. Widely used frameworks for building Generative AI agents such as Autogen [9], Taskweaver [10], and Semantic Kernel [11] adopt varied execution models, from collaborative multi-agent loops to planning-first architectures [12]. Despite the differences, these systems share a common modular structure comprising four key components: memory, planning, action and reflection [13,14] (see Fig. 1). These modules help interpret user intent, track execution history, and

* Corresponding author.

E-mail addresses: A.MatharaArachchige@latrobe.edu.au (A. Matharaarachchi), H.MORALIYAGE@latrobe.edu.au (H. Moraliyage), N.Mills@latrobe.edu.au (N. Mills), G.Gamage@latrobe.edu.au (G. Gamage), D.DeSilva@latrobe.edu.au (D. De Silva), misko@ieee.org (M. Manic).

<https://doi.org/10.1016/j.knosys.2026.115469>

Received 7 September 2025; Received in revised form 28 December 2025; Accepted 1 February 2026

Available online 7 February 2026

0950-7051/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

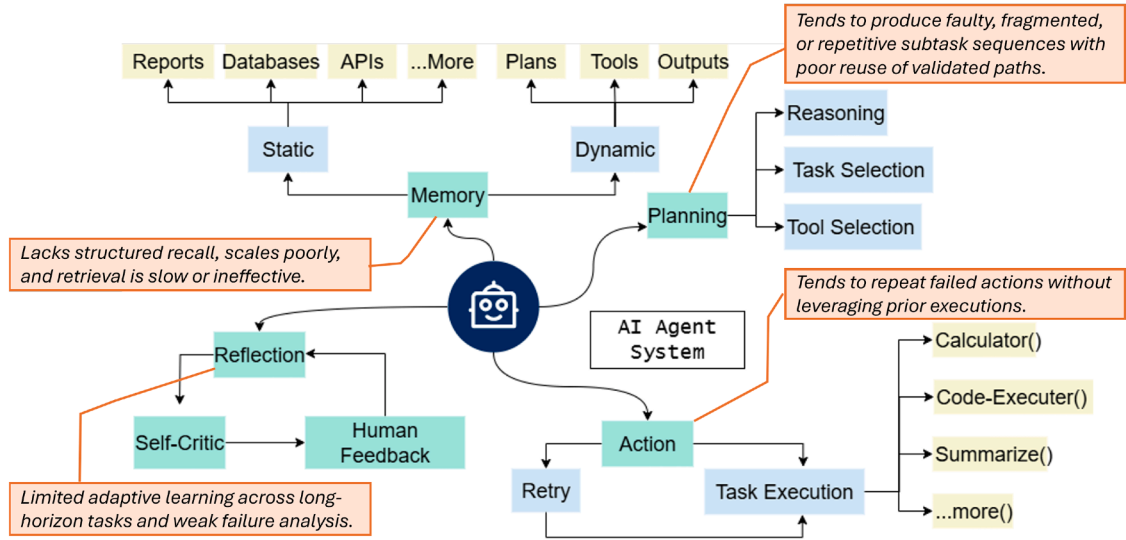


Fig. 1. Common modular structure of Generative AI agents composed of memory, planning, action and reflection.

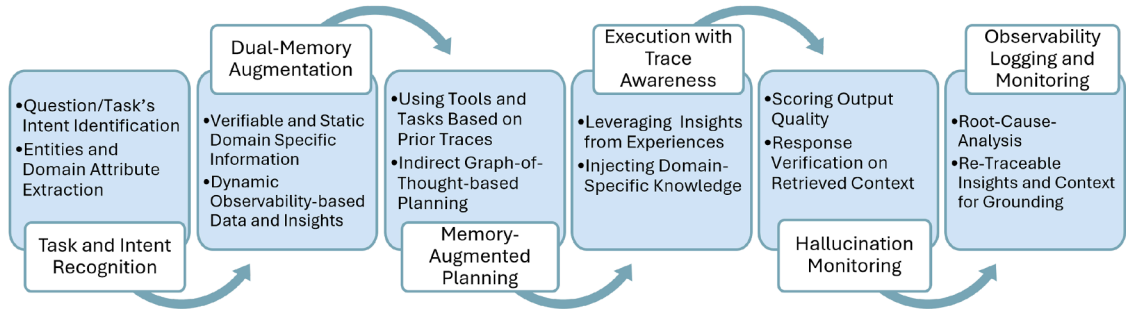


Fig. 2. Conceptual framing of the proposed dual memory knowledge graph approach for addressing hallucinations.

coordinate tool invocations. However, current implementations fail to retain semantically relevant memory, detect and avoid repeated failures, and maintain coherent long-term planning across interactions.

To address hallucinations, this paper draws on observability data contained in agent operation logs, tool calls, intermediate output, and errors already recognized in prior execution, as a foundation for structured memory and planning. This paper presents a dual memory knowledge graph approach based on semantic memory and observability memory for agents to recall validated behaviors, and leverage stored historical evidence during planning and generation. This approach transforms observability from passive monitoring into active memory that reduces hallucination. This approach is designed to complement existing reasoning frameworks such as ReAct and Reflexion. It provides a domain-agnostic memory layer that enhances these systems by supplying reusable validated traces and structured domain knowledge, allowing agents to operate with more grounded, and context-aware reasoning. Fig. 2 provides a conceptual framing of this approach where incoming queries are classified into intents, entities, and attributes, which anchor retrieval from both semantic and observability subgraphs.

This unified schema supports memory-augmented planning by linking current tasks to prior tool calls, outcomes, and validated planning paths. Based on recent work on structured memory architectures that distinguish episodic experiences from factual grounding [15,16], the approach further integrates a hallucination mitigation layer and a pruning mechanism to preserve a reliable and reusable memory store. Together, these components enable grounded, auditable, and self-improving agent behavior.

The rest of the paper is organized as follows. Section 2 presents related work in generative AI agents and addressing hallucinations,

followed by Section 3 presenting the dual memory knowledge graph approach, including knowledge graph design, query classification, memory construction, validation mechanisms and a case study demonstration. Section 4 presents a series of experiments evaluating the impact of observability memory on reasoning grounding, the contribution of semantic memory to factual grounding, the computational efficiency of the dual memory system, and the effectiveness of the validation pipeline. Section 5 concludes the paper.

2. Related work

Generative AI agents have progressed from single-turn prompt execution to modular systems capable of multi-step planning, tool invocation, and reflective reasoning. Frameworks such as Autogen [9] and Taskweaver [10] coordinate multiple agent roles through iterative dialogue and tool use. Despite these architectural advances, agents still hallucinate due to fragmented memory, weak factual grounding, and limited ability to reuse validated planning behaviours. As highlighted by Shah and White's critique "Agents Are Not Enough" [17], architectural modularity alone cannot overcome fundamental reasoning limitations; resilient systems require structured mechanisms that incorporate historical execution traces into the agent's decision process.

Retrieval-Augmented Generation (RAG) remains a widely adopted method for mitigating hallucinations. Traditional vector-based RAG [18] retrieves semantically similar text but often surfaces broad or partially relevant passages, particularly in multi-hop or compositional reasoning tasks, leading to context drift and factual errors. Recent surveys [19–21] note that vector-space retrieval struggles to preserve relational structure essential for precise grounding.

Table 1

Comparison of the proposed dual memory knowledge graph approach against related approaches.

Method	Trace Reuse	Ctx. Ground.	Reason. Ground.	Hallucination Type	Reduction Mechanism
GraphRAG [22]	–	✓	–	Factual	Structured graph retrieval
ODA [23]	–	✓	✓	Action/decision	Observation-driven action feedback
ReAct [27]	–	–	✓	Reasoning	Step-wise tool-informed reasoning
Reflexion [28]	–	–	✓	Reasoning	Self-critique of intermediate steps
Self-RAG [29]	–	✓	✓	Factual, reasoning	Retrieve, generate and critique loop
Toolformer [30]	–	–	✓	Tool misuse	Pretrained tool-use demonstrations
Ours (dual memory KG)	✓	✓	✓	Factual, reasoning	graph-based retrieval for context grounding, validated trace-based reasoning grounding

GraphRAG approaches [22] address these issues by organizing retrieved knowledge into structured subgraphs aligned with query semantics. This improves interpretability and supports more controlled grounding than vector methods. However, these techniques focus exclusively on external factual knowledge and do not incorporate execution traces, internal agent behaviour, or reusable planning histories. In contrast, our semantic memory forms only one half of a dual memory architecture, explicitly complemented by a structured observability memory that captures agent experience.

Knowledge graphs have been used for tasks such as contextual search and semantic retrieval, but more recent work incorporates KGs into reasoning and planning. Observation-Driven Agents (ODA) [23] employ KG-based feedback loops to inform action selection, demonstrating the value of structured observations during execution. Other KG-augmented methods explore knowledge-aware validation [24], graph-of-thought planning [25], or prompt tuning with retrieved triples [26].

Reflection-based methods, such as ReAct [27], Reflexion [28], and Self-RAG [29], reduce hallucinations by prompting models to critique or refine their intermediate reasoning steps. Similarly, Toolformer [30] improves tool usage through self-supervised tool-calling demonstrations. However, these techniques primarily employ ephemeral reflections: intermediate reasoning and tool outcomes are discarded after the session, and do not accumulate into a persistent memory structure.

Our approach extends reflection-based reasoning by transforming reflections, root-cause analyses, and execution outcomes into a persistent observability memory. This enables long-term reuse of validated planning trajectories and systematic avoidance of recurring failure patterns. This design is aligned with emerging work on using real-world agent logs for transparency and governance, such as Clio [31], but goes further by operationalizing logs as an active component of planning and reasoning rather than solely for auditing. The combination of semantic grounding and structured observability memory forms a dual memory approach that supports robust, trace-aware agent behaviour. A comparison of these approaches and our dual memory approach is provided in Table 2.

3. Methodology

This section presents the dual-memory knowledge graph approach in terms of its key functions of semantic memory focused on domain knowledge and observability memory based on agent execution traces, tool usage, and reflective insights. This structure grounds agent behavior in both external truth and internal experience. The proposed dual memory approach is agent-agnostic: it operates independently of the agent's internal prompting strategy, since it grounds reasoning through reusable observability traces and vocabulary-aligned semantic retrieval rather than relying on a specific decision-making algorithm.

As illustrated in Fig. 3, this agent-agnostic dual memory knowledge graph approach is generated and maintained through a modular structure consisting of four components. First, input data streams representing agent observability, including intermediate reasoning steps, tool invocations, outputs, and other application-level traces are pulled from observability storage logs. These logs are processed by the *Log Transformation Module*, which cleans, structures, and prunes them into reusable

reasoning experiences. Next, the Diagnostic Module identifies reasoning failures, performs root-cause analysis, and generates reflective insights to support adaptive plan revision. As part of this validation pipeline, knowledge-based verification and human-in-the-loop review are performed to distinguish successful runs from failed runs, yielding actionable insights for future planning. Afterward, the Classifier Module performs vocabulary-based tagging to align incoming traces, incorporating both insights from the Diagnostic Module and experiences from the *Log Transformation Module* with a shared ontology that is also used to annotate domain knowledge. These insights and experiences are then stored in the observability memory of the dual memory knowledge graph, enabling efficient retrieval and context-aware grounding for the AI agent's ongoing decision-making and planning processes. By integrating the dual memory into each agent stage (as observed in Fig. 1), the proposed approach enables traceable, verifiable, and agent planning with adaptive memory [15].

3.1. Knowledge graph design

Effective agent planning requires two distinct memory functions: access to verifiable external knowledge and reuse of past execution experience, and these represent the two complementary subgraphs: a semantic memory and an observability memory, unified through a shared vocabulary (Fig. 4).

The semantic memory encodes validated domain knowledge, including entities, attributes, and facts extracted from documents, databases, or APIs. It stores provenance details such as PDF locations, database tables, or URL references, enabling direct citation and traceable evidence during inference. This supports graph-based retrieval-augmented generation with high factual precision (Figs. 5–7).

The observability memory captures historical agent behavior, decision traces, and reflective outcomes, organized as linked plans and steps. This enables few-shot planning by retrieving successful past executions for similar query types and improves reasoning through insights gained from past reflections.

To unify these graphs at runtime, this paper adopts a shared vocabulary binding strategy. Each user query is classified into high-level labels (e.g., intent, entities, domain attributes), which serve as routing keys for both subgraphs. The same vocabulary schema is applied when ingesting domain facts into the semantic memory and when constructing nodes in the observability memory, so that both graphs expose a common label layer above their internal schemas. These labels are encoded into node annotations in the semantic memory and planning metadata in the observability memory, allowing the agent to retrieve both factual and experiential context through the same interface.

The observability KG nodes are stored independently of the semantic KG nodes in the same knowledge graph as two subgraphs, allowing each memory system to evolve at its own rate while maintaining data isolation. Although both graphs diverge in structure and update frequency, as the semantic memory is relatively static and the observability memory evolves over time, the shared vocabulary provides a lightweight alignment layer that allows heterogeneous sources to be associated through common labels rather than strict schema matching.

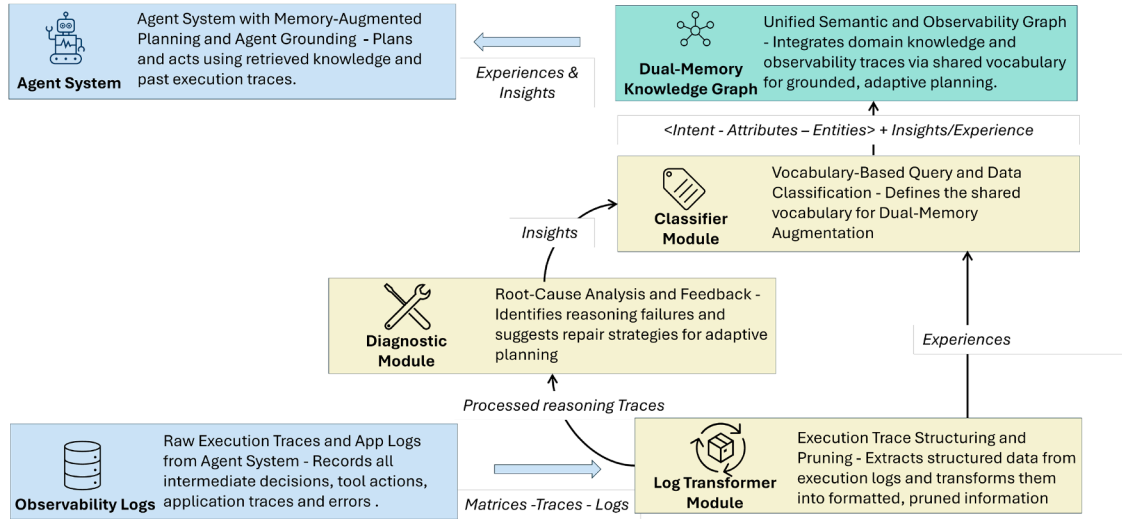


Fig. 3. Modular composition of the proposed dual memory knowledge graph approach.

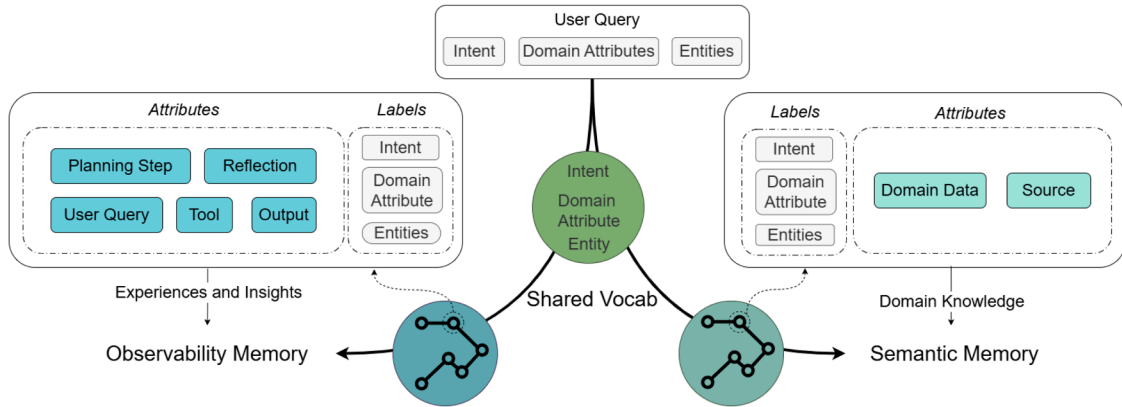


Fig. 4. Dual memory knowledge graph structure with shared vocabulary binding.

3.2. Vocabulary-Based query and data classification

To improve retrieval efficiency and planning precision, this approach employs a shared vocabulary classification mechanism applied at both query time and data ingestion. The Classification Module decomposes incoming content into this shared vocabulary. Since agent systems typically operate within a bounded domain, a fixed list of intents, attributes, and entity types is provided as soft constraints to improve consistency. In-context prompting avoids heavy fine-tuning and enables rapid adaptation to new domains. The resulting labels are embedded directly into graph nodes, enabling efficient and semantically aligned retrieval across all components of the agent system.

This mechanism ensures consistent annotation of user input, domain knowledge, and observability traces using structured labels: *intent*, *attributes*, and *entities*. These three facets constitute a compact coordination layer that enables alignment across heterogeneous sources without requiring brittle schema-level mappings.

The shared vocabulary provides a minimal, sufficient abstraction for linking the semantic memory and the observability memory: (1). *Intent*: captures the reasoning pattern required (e.g. lookup, comparison, multi-source aggregation) and guides which subgraphs or past experiences are relevant. (2). *Attributes*: describe coarse semantic types involved in the query (e.g. course, academic term, subject), enabling type-consistent alignment between domain content and past traces. (3). *Entities*: provide the concrete conceptual anchors mentioned in the query or trace (e.g. a specific course code, a named certificate program), allowing fine-grained retrieval and trace matching. Together, these labels act as stable

semantic join keys that unify structured domain knowledge and unstructured execution traces.

Importantly, defining the inventory of intents and domain attributes is a one-time setup step, usually performed by a human expert, reflecting the scope of the application domain. Once this vocabulary is established, no manual curation is required for individual queries or traces.

Based on recent work on unsupervised semantic clustering from large-scale dialogue data [31], a lightweight LLM-assisted classifier is used to extract these labels in three key scenarios:

1. Query-Time Classification.

Each query is mapped to a tuple $\langle \text{intent}, \text{attributes}, \text{entities} \rangle$, which serves as the routing key for retrieving relevant experiences and insights from the observability memory and routing key $\langle \text{attributes} \rangle$ for extracting relevant domain context from the semantic memory.

2. Source-Time Labeling.

During domain ingestion (e.g., reports, catalog entries, documents), the same schema for $\langle \text{intent}, \text{attributes}, \text{entities} \rangle$ is applied to tag items with the same vocabulary, ensuring that semantic content and observability traces are indexed under a unified representation.

3. Observability trace labeling.

As observability traces are processed by the *Log Transformation Module*, each validated experience and each diagnostic insight from observability traces is assigned its own $\langle \text{intent}, \text{attributes}, \text{entities} \rangle$ tuple before being persisted into the observability memory.

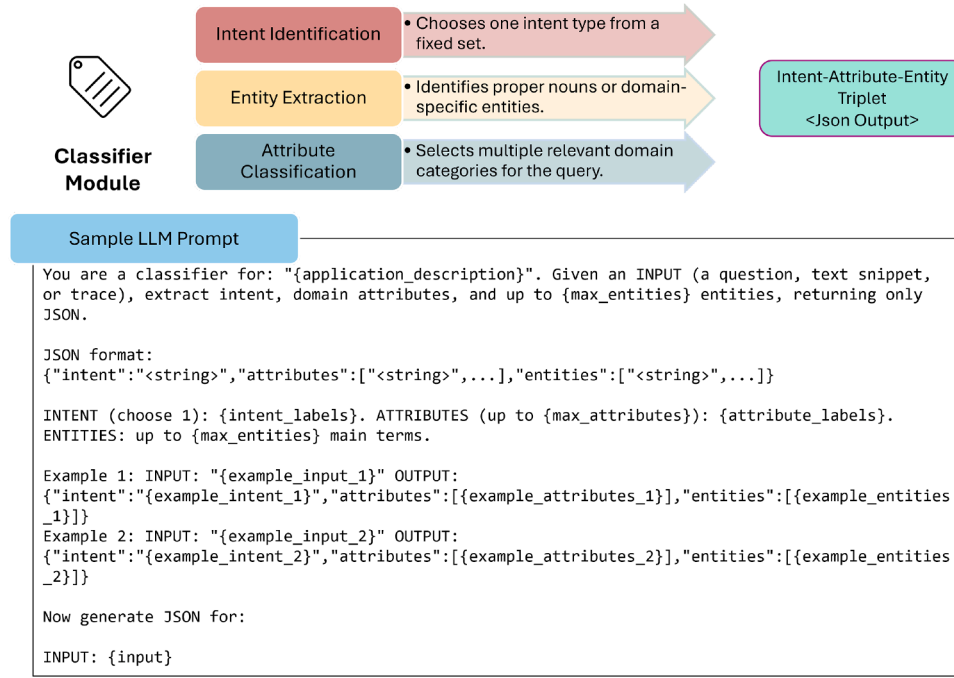


Fig. 5. Classifier module for structured semantic representation.

This unified labeling allows the system to dynamically annotate new queries and execution logs while maintaining alignment across heterogeneous sources. Each plan and step inherits its *<intent, attributes, entities>* labels, ensuring that both graphs expose the same label interface and can be referenced consistently during retrieval and planning.

3.3. Observability logging and graph-Based memory construction

To address reasoning-related hallucinations, the approach distinguishes between two forms of observability: a raw execution log and a structured observability memory. The raw log records the full sequence of the agent's actions and intermediate states, but it is used solely as an input source for extraction. Only the validated reasoning traces and synthesized diagnostic insights derived from these logs are retained in the observability memory.

AI observability tools typically export logs in a structured format and assign labels that distinguish reasoning-related records (e.g., model outputs, intermediate chains, tool calls) from infrastructure-level traces (See example in Fig. 6). The *Log Transformation Module* uses these labels to automatically isolate the portions of the log that correspond to the agent's reasoning process. This eliminates manual curation and permits a scalable, automated pipeline for transforming raw logs into structured observability memory.

Conceptually, the construction process of the observability memory consists of the following steps, as further depicted in Algorithm 1: Collecting execution traces from the AI observability tool, Extracting the reasoning traces of the agent, Performing diagnostic evaluation to identify successful and failed traces, Retaining verified successful traces as reusable planning examples, Storing failure-related insights separately for guidance and error prevention. Section 3.4 further describe how the methodology perform further diagnostic evaluation to identify successful and failed traces.

Only validated execution paths are stored as reusable plans, while failure nodes contribute diagnostic insight without influencing direct imitation. This dual treatment supports safe memory reuse and grounded future planning.

Algorithm 1 Observability-to-KG Transformation Pipeline.

```

1: procedure PROCESSLOGS(L)
2:   R ← FilterRelevant(L) ▷ retain only reasoning-related log entries
3:   T ← ExtractTrace(R) ▷ reconstruct task-specific reasoning trace
4:   D ← Diagnose(T) ▷ evaluate success/failure and extract insights
5:   if D.success = true then
6:     PersistExperience(T) ▷ store validated trace as reusable experience
7:   else
8:     PersistInsight(T, D) ▷ store as insight for plan reflections
9:   end if
10: end procedure

```

The observability process is formalized through a domain-agnostic schema. Table 2 defines the core ontology for logging decisions, reflections, and outcomes. Each observability trace is anchored by a unique run identifier (e.g., *q123_20240617T135402*), which is recorded across all graph components. Preserving this identifier across every knowledge graph node enables full trace reconstruction, even for discarded or failed branches, as long as raw logs are retained in archival storage. This design supports downstream auditing, failure replay, and root-cause analysis without bloating the observability memory.

These observability-based approach transform raw execution data into a reusable planning scaffold, allowing the agent to learn from its own planning history, not just static text.

3.3.1. Observability graph maintenance

The observability graph is maintained as a bounded memory rather than an accumulating log store. Only a curated set of recent, validated traces is retained as observability information is used for

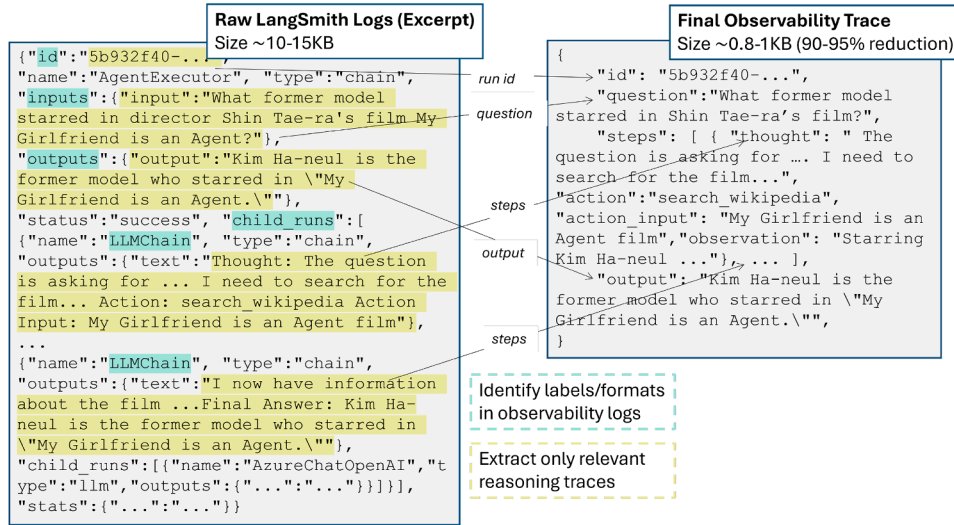


Fig. 6. Before/after comparison of raw observability logs and the cleaned reasoning trace used for observability memory construction.

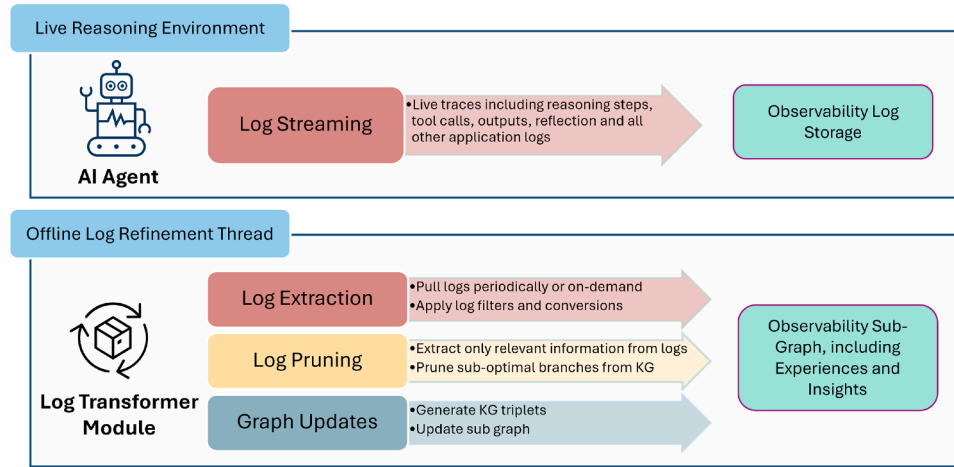


Fig. 7. Asynchronous Observability Log Pipeline where planning logs are streamed and later processed by the *Log Transformer Module* into reusable graph triplets.

Table 2

Core ontology for observability data with examples aligned to the education-domain eligibility case study.

Node Label	Key properties	Example instance
:Query	query tokens; user intent; embeddings; attributes - domain labels	{query: "Am I eligible for the diploma if I finished high school and met the English requirement?", intent: "eligibility_check", attributes: [{"program_requirements", "prerequisite", "score"}], entities: [{"diploma program", "English minimum score"]}}
:Plan	trace; insight; diagnostic signal; optimal $\in \{true, false\}$	{trace: "...", insight: "Ensure requirements come from the correct program; do not mix rules.", optimal: true}
:Step	type $\in \{retrieve, verify, compare, summarize\}$; status $\in \{ok, fail\}$; observation	{type: "retrieve", status: "ok", observation: "Diploma program requires completion of high school and minimum English score of 60.//}
:Tool	name; configuration; source type	{name: "program_requirements_api", config: "institution: AU_College", source: "institutional_policy_document"}
:Output	answer; hallucination score $\in [0, 1]$; human rating $\in [1, 5]$	{answer: "Yes, based on the published requirements, you meet the eligibility criteria.", hallucination score: 0.03, human rating: 4}

reasoning support. This method does not require frequent updates as periodic refreshes incorporating recent successful traces are sufficient to maintain planning stability. To control graph growth, the KG enforces strict node count limits. When the node limit is reached, the KG applies a selective sliding-window strategy: recent validated traces are retained, while older traces are considered for removal.

This process is not strictly chronological as the system preserves high-value traces, such as those with reliable human ratings and those frequently re-used in planning. Only redundant or low-utility traces are removed. This bounded-memory design keeps the observability graph lightweight, efficient, and aligned with the agent's evolving behavior.

3.4. Semantic graph organization

Recent literature consistently demonstrates that graph-based retrieval methods, such as GraphRAG [22], substantially outperform vector-based RAG in mitigating hallucinations [20], the present approach adopts graph traversal as the primary Retrieval-Augmented Generation (RAG) mechanism and does not attempt to propose alternatives or improvements to GraphRAG itself. Instead, the methodology adopts a domain-agnostic shared vocabulary that narrows the retrieval subgraph and improves relevance and latency. This design is motivated by the practical observation that domain applications are typically built around a finite set of well-defined sources; for example, in an education-domain system these may include program guides, institutional policies, accreditation documents or course catalogues. Each of these sources serves a well-defined intention: looking up degree information, identifying course requirements, or retrieving institutional policies. By encoding such intentions and semantic types explicitly through the shared vocabulary, the retrieval process is automatically restricted to the subgraph that matches the query pattern.

Before finalizing the base ontology, it is important to consider the downstream usage of the agent. To generalize across heterogeneous sources, the semantic memory organizes information using three elements: (1) *Statement* - factual domain data extracted from the source about an entity and (2) *Source* - provenance markers linking statements to their origin (e.g., documents, tables, APIs). (3) *<intent, attributes, entities>* - Statement is passed through the shared vocabulary classifier and assigned with *<intent, attributes, entities>* using *classification module*. These are connected using simple but extensible relation: (1) *Statement* $\rightarrow$ [SUPPORTED_BY] $\rightarrow$ *Source*. Domain Data are labelled with attributes and entities.

To achieve this integration, two storage strategies are available:

1. Full Materialization - all extracted facts are directly stored as graph nodes and edges, including data from PDFs, SQL tables, and APIs. This yields low-latency, single-phase RAG, but can lead to storage overhead.
2. Hybrid Materialization - only semantic highlights (entities, attributes, facets) are embedded in the graph. Source content remains in native repositories and is fetched on demand during inference. The KG stores pointers via stub nodes (e.g., *DocumentStub*, *SqlRowStub*, *ApiStub*).

In the hybrid approach, RAG proceeds in two stages: Phase 1: resolve graph pointers and extract source metadata. Phase 2: retrieve content from external repositories using context-specific extractors. While hybrid storage optimizes for scalability and data freshness, it demands additional handling, such as extractor modules, fallback logic for unavailable data, and circuit breakers for slow APIs. In contrast, full materialization simplifies planning and retrieval, but becomes impractical for large or frequently updated datasets. These strategies reflect trade-offs discussed in the broader literature on knowledge graph engineering, particularly with regard to balancing retrieval utility, storage cost, and update latency [32].

3.5. Validation pipeline for memory-augmented agent planning

Effective agent planning relies on two capabilities: reusing verified strategies and repairing failing ones. In the proposed system, these capabilities are orchestrated through a structured validation pipeline that evaluates intermediate outputs, diagnoses failures, and guides corrective actions after processing observability traces through the *Log Transformation Module*. This layer ensures that planning experiences and insights persisted in the observability graph remain reliable and aligned with quality standards.

The diagnostic pipeline consists of three cascading validation steps:

1. Root-Cause Analysis and Feedback - The *Diagnostic Module* evaluates observability traces processed by the *Log Transformation Module*. It

identifies whether a run succeeded, classifies failures using a predefined taxonomy (e.g., timeout, tool misuse, reasoning errors), and records a root cause and explanation. When a failure occurs, the *Diagnostic Module* synthesizes a short explanation and may recommend a repair strategy (Fig. 8). These insights are stored in the observability graph and later reused to guide agent reasoning.

2. Knowledge-Based Verification - Traces exhibiting missing or inconsistent responses relative to the retrieved context are marked as unusable for persistence in the observability memory. The verifier applies faithfulness-oriented hallucination detection metrics (Section 3) to evaluate factual consistency.
3. Human-in-the-Loop Review - For scenarios requiring expert oversight or where automated checks reach their limits, a Human-in-the-Loop (HITL) stage is required. In this stage, human evaluators assess the quality of a trace using a 1-5 rating scale based on criteria, such as correctness of intermediate reasoning, alignment between reasoning and final answer, and appropriateness of the solution. These ratings are persisted on *Output* nodes as *human_rating* properties. When multiple raters are involved, their scores are averaged, and traces with high rating variance may be flagged for further adjudication. the HITL stage is designed to reflect how oversight operates in real-world, large-scale systems. Rather than requiring exhaustive human review, the system supports selective or risk-aware validation, in which automated stages surface ambiguous or high-impact traces for prioritized human inspection. This enables expert judgment to be applied where it is most valuable while keeping the process scalable. On subsequent runs, plans with low average human scores are deprioritized even if their automatic markers are positive, ensuring that expert standards influence the search heuristic.

Only final answers that pass all validation stages are committed as reusable experiences in the observability memory, while failures are recorded as diagnostic insights. Over time, this process (1) biases planning toward higher-quality traces, (2) reduces reliance on trial-and-error, and (3) maintains an up-to-date model of what constitutes effective planning within the domain. Thus, diagnostic insights and verified experiences jointly bridge real-time planning with long-term memory optimization, enabling continual improvement of the agent's planning policy.

3.6. Demonstration

The proposed dual memory knowledge graph is demonstrated in this section for an AI agent operating in a tertiary education setting, providing responses to program-related queries. As illustrated in Fig. 9, the input user query is first processed by the Classification Module, which produces a shared-vocabulary tuple. Using the extracted *<intent, attributes, entities>* labels, the relevant domain context is extracted from the semantic memory. This step reduces factual hallucination by grounding the answer in structured, pre-existing knowledge rather than model-internal assumptions. The same vocabulary tuple is then used to retrieve past validated reasoning traces and diagnostics from the observability memory. These experiences supply more relevant reasoning patterns, reducing faithfulness hallucination, while the diagnostics provide reflective constraints (e.g., preventing the agent from presenting overlapping requirements across different programs). All retrieved facts can be tracked using observability traces to semantic KG nodes. The vocabulary tuple also makes routing decisions explicit, showing exactly why particular context was retrieved from semantic graph. Reasoning examples and insight nodes are similarly traceable, providing a transparent account of how the final answer was produced.

4. Experiments

We empirically evaluate the Dual-Memory Knowledge Graph across multiple dimensions to assess its impact on hallucination reduction.

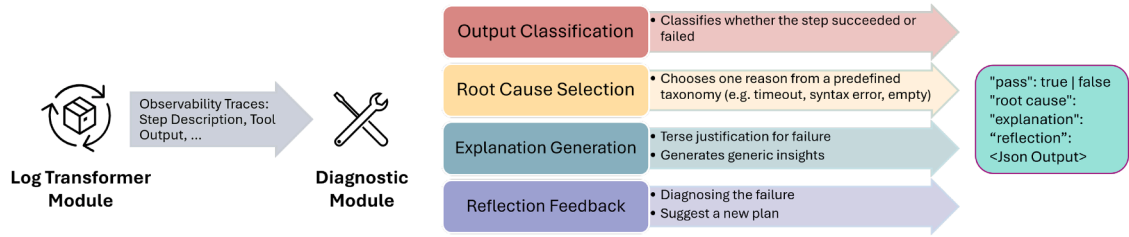


Fig. 8. Diagnostic Module for Root-cause analysis and feedback.

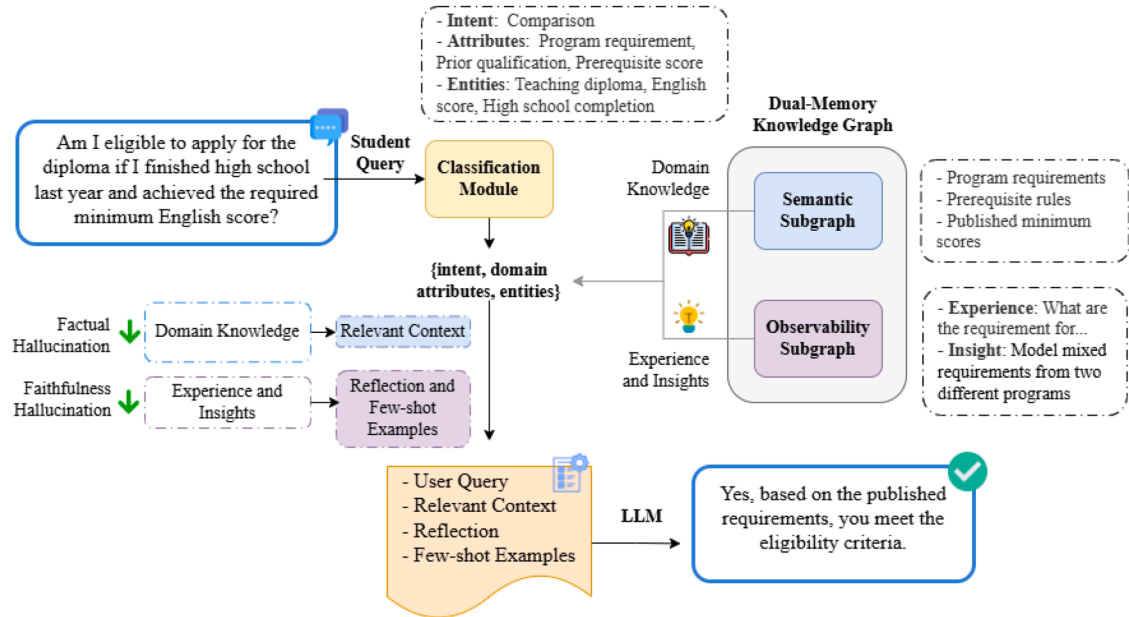


Fig. 9. End-to-end workflow of the dual memory knowledge graph in an education-domain advising query.

First, we examine how observability memory improves reasoning quality by comparing baseline agents with variants augmented with dual memory integration. Additionally, the semantic-memory experiment is included solely to demonstrate the behavior of vocabulary-conditioned subgraph retrieval and to verify that, in our setting, structured graph queries can retrieve focused factual context more efficiently than a simple vector-based RAG baseline. Furthermore, we quantify computational overhead, analyzing retrieval times, token usage, and number of LLM-calls introduced by the dual-memory design. Finally, we assess the effectiveness of the three-stage validation pipeline, demonstrating progressive improvements in filtering precision for reliable exclusion of erroneous traces. Collectively, these evaluations show that the dual-memory architecture enhances agent reliability while maintaining practical efficiency. Pre-processing pipelines, datasets, implementations, and evaluation scripts will be made available in our public GitHub repository, following the review process. <https://github.com/CDAC-lab/DualMemoryKG>

4.1. Evaluation metrics for hallucination reduction

We follow the hallucination taxonomy of Huang et al. [21], which divides hallucinations into two categories: Factuality Hallucination and Faithfulness Hallucination. In our evaluation, we used factuality metrics to measure the impact of the semantic memory, and faithfulness metrics were used to assess the observability memory impact. The complete list of metrics used to evaluate both the observability and semantic subgraphs is further explained in Table 3.

Given the lack of metrics that directly evaluate reasoning performance, we implemented custom DeepEval metrics named Instruction Inconsistency Score, Context Inconsistency Score, and Logical Inconsistency Score as depicted in Table 3. These metrics focus on evaluating the reasoning components of the observability subgraph. We also include the exact match metric, which evaluates whether the final response matches the dataset's gold answer and serves as a general accuracy measure across both experiments.

4.2. Stage 1: Evaluating the impact of observability memory

Observability memory was evaluated using both a ReAct-style agent [27] and a Reflexion agent [28] on the HotpotQA [34] dataset. A random subset of 100 instances from the dataset's training split was executed through the agents, and their raw execution logs were captured using the LangSmith observability platform [35]. These logs were processed asynchronously by the Log Transformation Module, which extracted reasoning traces and passed them through the Diagnostic Module for validation, as described in the methodology section. Only validated successful traces were persisted as reusable experiences in the observability memory, and diagnostic outputs from failed runs were stored as insights. All observability data was stored in a Neo4j Aura instance [36] using the ontology defined in Table 2.

To evaluate the contribution of reasoning grounding, we compared the baseline ReAct and Reflexion agents against versions augmented with observability-guided retrieval from the observability memory. Since observability memory primarily targets faithfulness hallucina-

Table 3

Mapping hallucination types from Huang et al. (2025) to evaluation metrics used in this work. All metrics are implemented using DeepEval [33].

Category	Metric	Explanation
Factuality Hallucination	Answer Relevancy Score	Measures the proportion of output statements judged relevant to the user query, indicating how well the final answer stays on topic; lower scores indicate irrelevant or off topic responses [33].
	Faithfulness Score	Measures how well the model's output aligns with the retrieved evidence by computing the proportion of generated claims that are truthful with respect to the retrieved context [33].
	Context Precision Score	The Contextual Precision metric evaluates the retriever by measuring how well it ranks relevant nodes above irrelevant ones, using a weighted cumulative precision that emphasizes correct ordering at the top of the retrieved context [33].
Faithfulness Hallucination	Instruction Inconsistency Score	Measures whether the answer adheres to the user instruction; deviations indicate instruction following failures.
	Context Inconsistency Score	Measures contradictions between reasoning steps or final answer and retrieved context.
	Logical Inconsistency Score	Evaluates internal logical coherence; higher scores indicate flawed or invalid reasoning steps.
	Bad Tool Calls	Fraction of tool calls that has failed due to errors due to incorrect tool names or wrong tool inputs, which indicates hallucinated reasoning.
	Total tool Calls	Total number of tool calls taken; excessive calls reflect inefficient reasoning steps.

Table 4

Hallucination metrics for ReAct and Reflexion agents, with and without observability-augmented knowledge.

Metric	ReAct	ReAct + Obs.KG	Reflexion	Reflexion + Obs.KG
Overall Performance (Higher value is better)				
Exact Match (Final Score)	0.59	0.63	0.63	0.70
Faithfulness / Reasoning Hallucination (Lower value is better)				
Instruction Inconsistency	0.53	0.52	0.43	0.42
Context Inconsistency	0.17	0.14	0.09	0.03
Logical Inconsistency	0.47	0.47	0.39	0.37
Bad Tool Calls	0.19	0.18	0.13	0.09
Total Tool Calls	1.71	1.65	2.14	2.11
Factuality Metrics (Higher value is better)				
Answer Relevancy Score	0.89	0.90	0.90	0.92
Faithfulness Score	0.74	0.81	0.85	0.90
Context Precision	0.53	0.53	0.58	0.65

tions, errors in reasoning steps, we report reasoning-oriented metrics as the main indicators of improvement. However, because stronger reasoning can indirectly reduce factual hallucinations, we also evaluate the factuality metrics to capture the secondary benefits of reasoning grounding.

For this experiment run, 200 random samples from the HotPotQA dataset's validation split were used, the full results are tabulated in Table 4.

Across both agent types, observability-guided reasoning yields consistent quantitative gains. Exact Match improves for ReAct (0.59→0.63) and more strongly for Reflexion (0.63→0.70), indicating better alignment of final responses with the dataset's gold answers. Reasoning also improves, most notably in context inconsistency, which drops from 0.17 to 0.14 for ReAct and sharply from 0.09 to 0.03 for Reflexion. Instruction inconsistency shows smaller but consistent decreases across both settings. For logical inconsistency, Reflexion shows a modest improvement (0.39→0.37), while ReAct remains unchanged. This indicates that observability memory either improves or preserves logical coherence depending on the agent architecture, with stronger benefits observed when combined with reflective reasoning mechanisms. Tool-use quality also improves, with bad tool calls reduced for both agents, marginally for

ReAct (0.19→0.18) and more substantially for Reflexion (0.13→0.09), alongside slight reductions in total tool calls. In the HotpotQA setting, fewer total tool calls indicate more precise and targeted tool usage, as successful reasoning requires retrieving the correct supporting facts with minimal intermediate steps. Thus, reductions in total tool calls reflect improved reasoning efficiency.

Factuality metrics showed steady secondary benefits from improved reasoning. Answer relevancy increased for both agents (ReAct: 0.89→0.90, Reflexion: 0.90→0.92), and faithfulness improved from 0.74 to 0.81 for ReAct and from 0.85 to 0.90 for Reflexion. Context precision also increased across settings (ReAct remained stable; Reflexion: 0.58→0.65 improved), indicating that more accurate reasoning steps translate into more reliable factual grounding.

Overall, these results demonstrate that the proposed dual memory knowledge graph framework with validated observability traces significantly enhances agent reliability by producing fewer reasoning errors, making better-informed tool calls, and delivering more accurate final answers. These gains highlight the practical value of observability not only as diagnostic tools but as an integrated reasoning substrate that strengthens both reasoning and factual grounding in modern AI agents.

Table 5
Results of factuality hallucination metrics.

Method	Latency (ms)	Success (%)	Relevancy (%)	Faithfulness (%)	Precision (%)
Vector-only QnA	4485.16	56.04	94.50	97.80	59.34
Microsoft GraphRAG	55487.26	15.55	58.24	96.29	22.08
Graph-based QnA (Ours)	3884.16	86.81	97.80	99.78	87.91

4.3. Stage 2: Evaluating semantic memory

This experiment evaluates semantic memory for factual grounding, comparing it against a vector-RAG baseline. All queries were first processed by the Vocabulary-Based Classifier Module, which generated structured tuples of intent, entity, and attribute to guide precise sub-graph retrieval. The experiment uses the SM3-Text-to-Query benchmark [37], built on the Synthea synthetic patient dataset [38], to create a question, cyphers and answer set for evaluation. Initially, subject-predicate-object triples from Synthea dataset were extracted to construct a Neo4j knowledge graph [39], resulting in a final graph, including: 8,516 nodes and 150,675 relationships. This graph was used to retrieve contextual information via Cypher queries, which were generated dynamically based on the user's question. The retrieved results are injected as context into the LLM prompt. As the baseline, a vector-RAG model which uses the same underlying data is embedded using a SentenceTransformer model. Given a query, vector search is used to retrieve top-k similar snippets, which are then passed to the LLM as context. Both systems share the same summarization model, GPT-4o [40] and prompt template to ensure a controlled comparison.

In addition to these baselines, we also performed an evaluation using Microsoft GraphRAG. Although GraphRAG is designed for broad knowledge exploration and summarization like tasks, we included it here to show that a simple, domain-anchored graph retrieval workflow can be more effective for structured domain tasks that require precise, localized information. To enable GraphRAG processing, the Synthea triples were converted into a text-based corpus following the library requirements. We selected *local search* mode because the SM3 questions target localized, entity-level facts, whereas *global search* returned broader summaries that aligned less with these queries.

Each system was evaluated using the factuality hallucination metrics in 3. The results of this experiment are presented in Table 5. Graph-based retrieval outperformed vector-RAG in both factual alignment and response relevance. The latency for Cypher queries was lower on average than the full vector search, likely because Cypher returns precise, targeted results, while vector-RAG retrieves longer, less focused documents that increase prompt size and LLM processing time. Graph-based QnA outperformed vector search in the answer relevancy score, indicating that structured retrieval yields more on-target concise answers. Although both methods achieved high faithfulness scores, the graph-based approach further reduced residual hallucinations compared to vector-only retrieval. The graph-based system showed stronger precision compared to vector-only retrieval. It reflects fewer hallucinations and tighter contextual alignment when using graph-derived facts. This confirms that even when backed by the same language model, the semantic retrieval structure alone plays a crucial role in hallucination suppression. Furthermore, GraphRAG showed higher latency and lower relevancy and precision in this domain setting, reflecting differences in retrieval granularity and the structure of the input data. Overall, the graph-based QnA system demonstrated fewer hallucinations and tighter contextual grounding across all factuality metrics. These results show that, even when using the same underlying language model, a vocabulary-guided simple graph retrieval workflow can better support structured domain queries that rely on precise, entity-level context.

4.4. Computational cost and efficiency analysis

This section evaluates the computational cost of augmenting ReAct and Reflexion agents with the proposed dual memory knowledge graph approach. Since graph construction occurs offline, three online operations contribute towards computational overhead: (1) vocabulary classification, (2) semantic context retrieval, and (3) reuse of validated observability traces.

As shown in Fig. 10, the dual-memory KG-augmented ReAct agent reduces agent execution latency from 9.10 s to 7.44 s, while maintaining comparable total latency (9.16 s vs. 9.66 s). Token usage is modestly reduced, with total tokens decreasing from 2346.68 to 2326.34, and LLM-call frequency remaining effectively unchanged (3.05 vs. 3.06). A similar but more pronounced trend is observed for the Reflexion agent, where agent latency decreases from 25.91 s to 23.51 s and total token usage drops substantially from 7321.04 to 6529.34, alongside a reduction in LLM calls (7.59 to 7.40). Across both agent families, the modest overhead introduced by observability-guided retrieval is largely offset by reductions in reasoning steps, resulting in improved execution efficiency without increasing inference-time cost.

As shown in Fig. 11, retrieval latency remains the same at approximately 0.46 ms across KG sizes ranging from 100 to 1000 nodes, indicating that query performance is effectively constant with respect to graph growth. As noted in the methodology Section 3.3.1, the persisted trace count does not grow beyond this range because the system maintains a sliding-window, bounded graph rather than a full historical store. The combined cost of vocabulary classification and trace reuse contributes an average latency of 2.5 s per query as shown in Fig. 10.

Additionally, pruning and diagnostic filtering compressed stored traces to 1 KB, compared to 15 KB for raw observability logs (> 90% reduction), confirming the compactness of the observability memory representation. While performance varies with domain size, database configuration, and model latency. Collectively, these results confirm that Observability memory integration imposes minimal computational cost while improving execution efficiency through more targeted tool-use and trace-guided reasoning.

4.5. Evaluating validation pipeline efficiency

The three-stage validation pipeline is evaluated using a student-advising agent operating in an education domain as presented in the Demonstration Section 3.6. Stage 3 introduces a Human-in-the-Loop (HIL) review to ensure that only reliable traces are added to the observability memory. In this step, four evaluators with backgrounds in higher-education advising and student-support services rated each processed trace including the user question, the agent's intermediate reasoning steps, and the final answer on a 1-5 quality scale. Inter-rater agreement was measured using Fleiss' Kappa, yielding a value of 0.71, indicating substantial agreement. For each trace, the final score was computed as the average of all four ratings, and traces below a preset threshold of 3 were rejected. Disagreements were resolved via majority vote, with unresolved cases escalated to a fifth adjudicator.

Each stage operates only on traces that were marked as successful by the previous one, producing a progressively smaller and more reliable candidate set. Because the objective is to prevent incorrect traces from entering the KG, the pipeline is designed to prioritize precision over

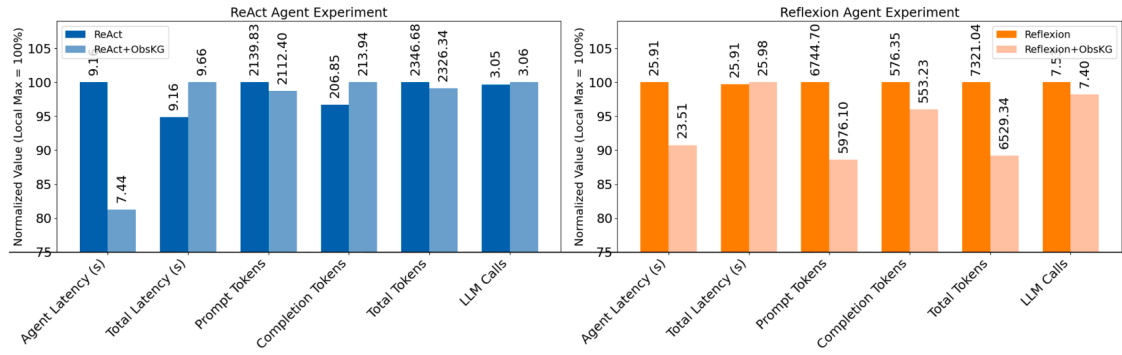


Fig. 10. Normalized computational efficiency of baseline agents versus their dual KG-augmented variants. Each subplot is normalized within model family, with raw values shown above bars.

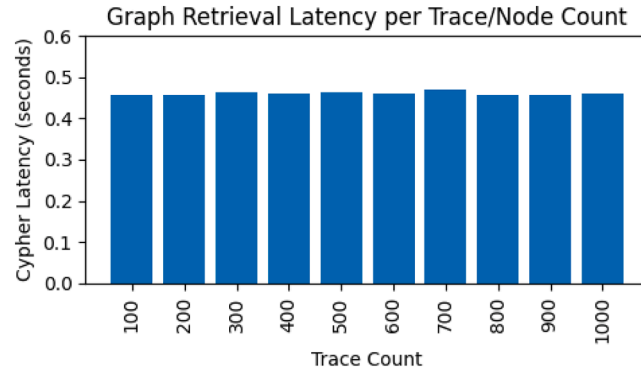


Fig. 11. Retrieval latency of observability memory for node growth between 100–1000 nodes.

Table 6

Confusion matrix across the three validation stages.

Stage	TP	FP	FN	TN	FDR	Precision
Root-Cause Analysis	33	12	16	39	26.7%	73.3%
Knowledge Verification	18	5	15	7	21.7%	78.3%
Human Review	17	2	1	3	10.5%	89.5%

recall: reducing false positives(FP) is essential, while false negatives(FN) are acceptable since discarding uncertain traces is safer than retaining potentially incorrect ones.

To interpret the validation outcomes, we use a standard confusion-matrix: *True Positives (TP)* denote valid traces correctly accepted, and *True Negatives (TN)* are invalid traces correctly rejected. *False Positives (FP)* represent invalid traces mistakenly accepted, whereas *False Negatives (FN)* refer to valid traces incorrectly rejected. Two derived measures are central to evaluating filtering effectiveness: *precision*, the fraction of accepted traces that are correct ($\text{Precision} = \frac{TP}{TP+FP}$), and the *false discovery rate (FDR)*, the fraction of accepted traces that are incorrect ($\text{FDR} = \frac{FP}{TN+FP}$). Lower FDR and higher precision indicate stronger filtering of erroneous traces.

As shown in Table 6, the three stages exhibit a clear improvement in these measures. Root-Cause Analysis provides broad initial filtering; Knowledge-Based Verification substantially reduces incorrect acceptances; and Human Review resolves the remaining ambiguous cases with the highest precision. This cascading procedure ensures that only traces with strong evidence of correctness progress through the pipeline.

5. Conclusion

This paper presented a dual memory agent approach that integrates semantic knowledge with observability-driven execution traces to im-

prove factual and reasoning grounding, reducing hallucinations in Generative AI agents. Experimental results demonstrate that graph-based retrieval outperforms traditional vector-based approaches in hallucination mitigation. Likewise, observability-guided planning improves agent planning efficiency and robustness in complex reasoning tasks, reducing both faithfulness and factual hallucinations. Generative AI agents in this study are primarily text-based, for tasks such as code generation and API interaction. Extension to multimodal agent systems is planned as future work. Although fine-tuning or architectural modifications may yield further improvements, this work prioritizes in-context learning techniques such as prompt-based exemplars and retrieval-augmented generation for integrating historical planning flows and responses due to interpretability and ease of integration with pre-existing agents. Further developments of this work include advancing the shared vocabulary to reduce reliance on human-designed heuristics and improving robustness of semantic memory by integrating generalized graph-retrieval techniques. Additional benchmark development is also needed to assess planning reuse, error recovery, and cross-task generalization within agent frameworks.

CRedit authorship contribution statement

Amali Matharaarachchi: Writing – original draft, Validation, Methodology, Investigation, Formal analysis; **Harsha Moraliyage:** Writing – original draft, Validation, Supervision, Methodology, Investigation; **Nishan Mills:** Writing – original draft, Software, Methodology, Investigation, Formal analysis; **Gihan Gamage:** Writing – original draft, Methodology, Investigation, Formal analysis, Conceptualization; **Daswin De Silva:** Writing – original draft, Supervision, Methodology, Investigation, Conceptualization; **Milos Manic:** Writing – original draft, Supervision, Methodology, Investigation, Conceptualization.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work is supported by the [Department of Climate Change, Energy, the Environment and Water](#) of the Australian Federal Government, as part of grant program ICIRN000077.

References

- [1] A. Zou, Y. Chen, R. Huang, Y. Qin, A resilient generative model in few-shot question answering, *Knowl. Based Syst.* (2025) 113629.
- [2] D. De Silva, O. Kaynak, M. El-Ayoubi, N. Mills, D. Alahakoon, M. Manic, Opportunities and challenges of generative artificial intelligence: research, education, industry engagement, and social impact, *IEEE Ind. Electron. Mag.* (2024).
- [3] M. Zaib, W.E. Zhang, Q.Z. Sheng, A. Mahmood, Y. Zhang, Conversational question answering: a survey, *Knowl. Inf. Syst.* 64 (12) (2022) 3151–3195.
- [4] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y.J. Bang, A. Madotto, P. Fung, Survey of hallucination in natural language generation, *ACM Comput. Surv.* 55 (12) (2023) 1–38.
- [5] Y. Kim, H. Jeong, S. Chen, S.S. Li, M. Lu, K. Alhamoud, J. Mun, C. Grau, M. Jung, R. Gameiro, et al., Medical hallucinations in foundation models and their impact on healthcare, (2025). [arXiv preprint arXiv:2503.05777](#)
- [6] S.L. Mirtaheri, A. Pugliese, V. Pascucci, et al., Automated vulnerability score prediction through lightweight generative AI, *Knowl. Based Syst.* (2025).
- [7] M.-K. Ghali, A. Farrag, D. Won, Y. Jin, Enhancing knowledge retrieval with in-context learning and semantic search through generative AI, *Knowl. Based Syst.* 311 (2025) 113047.
- [8] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N.V. Chawla, O. Wiest, X. Zhang, Large language model based multi-agents: A survey of progress and challenges, (2024). [arXiv preprint arXiv:2402.01680](#)
- [9] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, C. Wang, Autogen: Enabling next-gen llm applications via multi-agent conversation framework, 3 (4) (2023). [arXiv preprint arXiv:2308.08155](#)
- [10] B. Qiao, L. Li, X. Zhang, S. He, Y. Kang, C. Zhang, F. Yang, H. Dong, J. Zhang, L. Wang, M. Ma, P. Zhao, S. Qin, X. Qin, C. Du, Y. Xu, Q. Lin, S. Rajmohan, D. Zhang, TaskWeaver: A Code-First Agent Framework, (2023). [arXiv preprint arXiv:2311.17541](#)
- [11] Microsoft, Semantic Kernel: Open-Source SDK for Building AI Agents, 2023. Accessed: 2025-07-09, <https://github.com/microsoft/semantic-kernel>.
- [12] G. Gamage, S. Kahawala, N. Mills, D. De Silva, M. Manic, D. Alahakoon, A. Jennings, Augmenting industrial chatbots in energy systems using chatgpt generative ai, in: 2023 IEEE 32nd International Symposium on Industrial Electronics (ISIE), IEEE, 2023, pp. 1–6.
- [13] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al., A survey on large language model based autonomous agents, *Front. Comput. Sci.* 18 (6) (2024) 186345.
- [14] L. Weng, LLM-powered Autonomous Agents, 2023. Blog post, <https://lilianweng.github.io/posts/2023-06-23-agent/>.
- [15] C. de Masson D'Autume, S. Ruder, L. Kong, D. Yogatama, Episodic memory in life-long language learning, *Adv. Neural Inf. Process. Syst.* 32 (2019).
- [16] W. Wang, L. Dong, H. Cheng, X. Liu, X. Yan, J. Gao, F. Wei, Augmenting language models with long-term memory, *Adv. Neural Inf. Process. Syst.* 36 (2023) 74530–74543.
- [17] C. Shah, R.W. White, Agents are not enough, (2024). [arXiv preprint arXiv:2412.16241](#)
- [18] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, in: *Advances in Neural Information Processing Systems*, 33, 2020, pp. 9459–9474.
- [19] B. Peng, Y. Zhu, Y. Liu, X. Bo, H. Shi, C. Hong, Y. Zhang, S. Tang, Graph retrieval-augmented generation: A survey, (2024). [arXiv preprint arXiv:2408.08921](#)
- [20] G. Agrawal, T. Kumarage, Z. Alghamdi, H. Liu, Can knowledge graphs reduce hallucinations in llms?: A survey, (2023). [arXiv preprint arXiv:2311.07914](#)
- [21] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, et al., A survey on hallucination in large language models: principles, taxonomy, challenges, and open questions, *ACM Trans. Inf. Syst.* 43 (2) (2025) 1–55.
- [22] M. Research, GraphRAG: Graph Retrieval-Augmented Generation, 2024, (<https://microsoft.github.io/graphrag/>). Accessed: 2025-06-27.
- [23] L. Sun, Z. Tao, Y. Li, H. Arakawa, Oda: Observation-driven agent for integrating llms and knowledge graphs, (2024). [arXiv preprint arXiv:2404.07677](#)
- [24] A. De Santis, M. Balduini, F. De Santis, A. Proia, A. Leo, M. Brambilla, E. Della Valle, Integrating large language models and knowledge graphs for extraction and validation of textual test data, in: *International Semantic Web Conference*, Springer, 2024, pp. 304–323.
- [25] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, et al., Graph of thoughts: solving elaborate problems with large language models, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38, No. 16, 2024, pp. 17682–17690.
- [26] D. Cheng, S. Huang, J. Bi, Y. Zhan, J. Liu, Y. Wang, H. Sun, F. Wei, D. Deng, Q. Zhang, Uprise: Universal prompt retrieval for improving zero-shot evaluation, (2023). [arXiv preprint arXiv:2303.08518](#)
- [27] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, Y. Cao, React: synergizing reasoning and acting in language models, in: *International Conference on Learning Representations (ICLR)*, 2023.
- [28] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao, Reflexion: language agents with verbal reinforcement learning, in: *Advances in Neural Information Processing Systems*, 36, 2023, pp. 8634–8652.
- [29] A. Asai, Z. Wu, Y. Wang, A. Sil, H.S.-R. Hajishirzi, learning to retrieve, generate, and critique through self-reflection. (2023). [arXiv. 10.48550, arXiv preprint arXiv:2310.11511](#)
- [30] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: language models can teach themselves to use tools, *Adv. Neural Inf. Process. Syst.* 36 (2023) 68539–68551.
- [31] A. Tamkin, M. McCain, K. Handa, E. Durmus, L. Lovitt, A. Rath, S. Huang, A. Mountfield, J. Hong, S. Ritchie, et al., Chio: Privacy-Preserving Insights into Real-World AI Use, (2024). [arXiv preprint arXiv:2412.13678](#)
- [32] S. Ji, S. Pan, E. Cambria, P. Marttinen, S.Y. Philip, A survey on knowledge graphs: representation, acquisition, and applications, *IEEE Trans. Neural Netw. Learn. Syst.* 33 (2) (2021) 494–514.
- [33] A.I. Confident, DeepEval: The Open-Source LLM Evaluation Framework, 2025, (<https://deepeval.com>). Version 1.0.0. Available at <https://github.com/confident-ai/deepeval>.
- [34] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W.W. Cohen, R. Salakhutdinov, C.D. Manning, HotpotQA: A dataset for diverse, explainable multi-hop question answering, (2018). [arXiv preprint arXiv:1809.09600](#)
- [35] LangChain Technologies, LangSmith: A Platform for LLM Application Evaluation and Observability, 2023. <https://smith.langchain.com>.
- [36] Neo4j, Inc., Neo4j Aura: Fully Managed Graph Database as a Service, 2020, <https://neo4j.com/cloud/aura>.
- [37] S. Sivasubramaniam, C. Osei-Akoto, Y. Zhang, K. Stockinger, J. Fuerst, SM3-Text-to-Query: Synthetic Multi-Model Medical Text-to-Query Benchmark, 2024, <https://arxiv.org/abs/2411.05521>.
- [38] J. Walonoski, M. Kramer, J. Nichols, A. Quina, C. Moesel, D. Hall, C. Duffett, K. Dube, T. Gallagher, S. McLachlan, Synthesia: an approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record, *J. Am. Med. Assoc.* 25 (3) (2018) 230–238.
- [39] GPTechday, OpenAI Academy Knowledge Graph Recipe, 2025, (<https://github.com/gptechday/openai-academy-kg-recipe>). Accessed: 2025-06-29.
- [40] OpenAI, GPT-4o Technical Report, 2024, (<https://openai.com/research/gpt-4o>). Accessed: 2025-06-28.